**Міністерство освіти і науки України**

**Львівський національний університет імені Івана Франка Факультет електроніки та комп’ютерних технологій**

**Звіт**

Про виконання лабораторної роботи № 8

**«МАСИВИ»**

Виконав:

студент 1 курсу

групи ФЕП-11с

**Ребрик Яник**

Викладач:

асист. Васюта В. М.

**Львів 2026**

## Мета роботи

Навчитися створювати й обробляти одновимірні та двовимірні масиви в мові C, передавати їх у функції та отримувати адреси елементів.

## 2. Масив із 10 випадкових чисел

Функція fillArray заповнює масив числами від 0 до 99 і повертає адресу першого елемента. Функція showArray виводить індекс і значення кожного елемента.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 10

int *fillArray(int numbers[])
{
    for (int i = 0; i < SIZE; i++) {
        numbers[i] = rand() % 100;
    }

    return numbers;
}

void showArray(const int numbers[])
{
    for (int i = 0; i < SIZE; i++) {
        printf("numbers[%d] = %d\n", i, numbers[i]);
    }
}

int main(void)
{
    int numbers[SIZE];

    srand((unsigned int)time(NULL));

    int *filledArray = fillArray(numbers);
    showArray(filledArray);

    return 0;
}
```

## 3. Вказівники на елементи масивів

Функція arrayAddress повертає адресу елемента одновимірного масиву за заданим індексом. Функція matrixAddress повертає адресу елемента матриці 12 × 12 або NULL, якщо індекс рядка чи стовпця виходить за межі.

```c
#include <stdio.h>

#define SIZE 10
#define SIDE 12

int *arrayAddress(int mas[], int index)
{
    return &mas[index];
}

int *matrixAddress(int matrix[][SIDE], int row, int col)
{
    if (row < 0 || row >= SIDE || col < 0 || col >= SIDE) {
        return NULL;
    }

    return &matrix[row][col];
}

int main(void)
{
    int mas[SIZE];
    int matrix[SIDE][SIDE];

    int *first = arrayAddress(mas, 5);
    printf("Array address: %p\n", (void *)first);

    int *second = matrixAddress(matrix, 5, 7);
    if (second != NULL) {
        printf("Matrix address: %p\n", (void *)second);
    } else {
        printf("Matrix element not found.\n");
    }

    return 0;
}
```

## 4. Мінімум, максимум та їхня сума

Функція setArray заповнює масив із 20 чисел у діапазоні від 0 до 99. Функція searchElement знаходить найменше та найбільше значення за один прохід і записує їхню суму.

```c
#define SIZE 20

int randomValue(void)
{
    return rand() % 100;
}

void setArray(int mas[])
{
    for (int i = 0; i < SIZE; i++) {
        mas[i] = randomValue();
    }
}

void searchElement(const int mas[], int *sum, int *min, int *max)
{
    *min = mas[0];
    *max = mas[0];

    for (int i = 1; i < SIZE; i++) {
        if (mas[i] < *min) {
            *min = mas[i];
        }

        if (mas[i] > *max) {
            *max = mas[i];
        }
    }

    *sum = *min + *max;
}

int main(void)
{
    int mas[SIZE];
    int sum, min, max;

    srand((unsigned int)time(NULL));
    setArray(mas);
    searchElement(mas, &sum, &min, &max);

    printf("Min: %d, Max: %d, Sum: %d\n", min, max, sum);

    return 0;
}
```

## Висновок

У роботі використано масиви для зберігання випадкових чисел і вказівники для отримання адрес елементів та передавання результатів із функції. Реалізовано пошук мінімуму, максимуму й обчислення їхньої суми.